

Deep Learning Assignment 1

Gradient Descent and CIFAR-100 Image Classification

Mohamad Rami Karkour

mokr22@student.bth.se

Question 1 - Optimization using gradient descent

The function I worked with in the first part was $f(x,y) = (x - 3)^2 + (y + 2)^2$. Since the function is just a sum of two squared terms, the minimum can also be found directly by looking at where each squared term becomes zero. That happens when $x = 3$ and $y = -2$.

For the derivative with respect to x , y is treated as constant. This gives $df/dx = 2(x - 3)$. For the derivative with respect to y , x is treated as constant, so $df/dy = 2(y + 2)$. The gradient used in the code was therefore $[2(x - 3), 2(y + 2)]$.

I started the gradient descent at $(0, 0)$. The algorithm moved in the negative gradient direction and gradually approached the theoretical minimum. The final point from the program was approximately $(2.999984, -1.999989)$, with a final loss of 0.0000000004 . This is not exactly $(3, -2)$, but it is close enough to show that the method converged correctly. The small difference comes from numerical iteration.

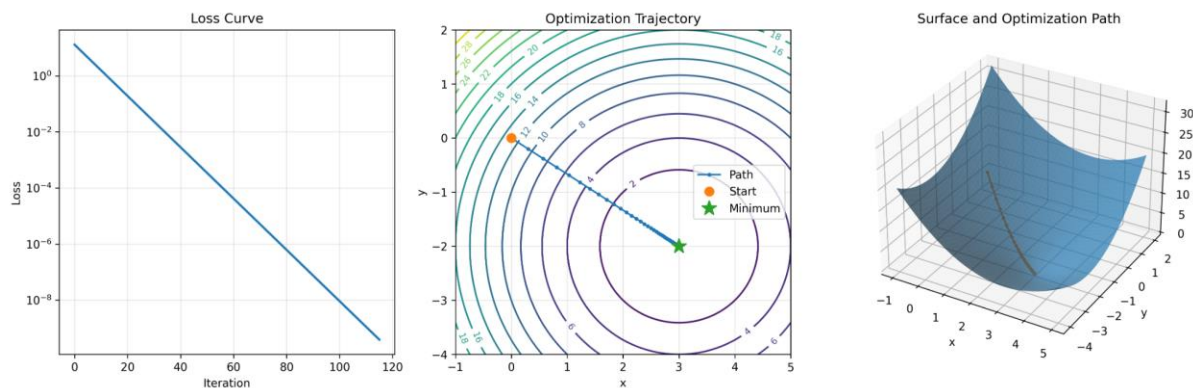


Figure 1. Loss curve, 2D trajectory, and 3D surface for the gradient descent run.

Question 2 - CIFAR-100 image classification

For the second part, I used the CIFAR-100 dataset through `torchvision.datasets.CIFAR100`. The dataset has 100 classes, which makes the task much harder than CIFAR-10. I used a train, validation, and test setup. The official training set was split into training and validation data, while the official test set was saved for the final evaluation.

Custom CNN architecture

I built a custom convolutional network using a simple block structure: convolution, batch normalization, ReLU, and max pooling in several places. I wanted the model to be strong enough for CIFAR-100 without making it too heavy to train. The number of channels increases as the image goes deeper into the network, from 64 channels in the first layer up to 512 channels in the deeper layers.

Stage	Main operation	Output channels	Kernel/stride/padding
1	Conv + BatchNorm + ReLU + MaxPool	64	3x3 / 1 / 1
2	Conv + BatchNorm + ReLU + MaxPool	128	3x3 / 1 / 1
3	Conv + BatchNorm + ReLU	256	3x3 / 1 / 1

4	Conv + BatchNorm + ReLU + MaxPool	256	3x3 / 1 / 1
5	Conv + BatchNorm + ReLU	512	3x3 / 1 / 1
6	Conv + BatchNorm + ReLU + MaxPool	512	3x3 / 1 / 1
Classifier	Flatten + Linear layers + Dropout	100 classes	-

Training setup and preprocessing

The custom CNN was trained from scratch. For all models, I used cross-entropy loss since this is a multi-class classification problem. The optimizer was AdamW with weight decay, and I used a cosine learning-rate schedule. I also used mixed precision training because it makes training faster on the GPU (my old RTX 3070) without changing the main idea of the experiment.

For preprocessing, I kept the CIFAR-100 images at their original 32x32 resolution. For training data, I used random crop, random horizontal flip, random rotation, and color jitter. Validation and test data were only normalized.

I also fine-tuned ResNet-50, VGG-19 with batch normalization, DenseNet-121, and EfficientNet-B0. The pretrained models used ImageNet weights as a starting point. I expected these models to do better than the custom CNN because they already learned useful image features before being trained on CIFAR-100.

Training analysis

The training curves show that the models were learning properly. The losses mostly went down over time, and the accuracies went up. The custom CNN improved slowly but steadily over 40 epochs. The pretrained models improved much faster in the first few epochs.

Early stopping logic was included in the training code, but in this run the models completed their planned epochs. There is some gap between training and validation accuracy for the pretrained models, especially VGG-19 and ResNet-50, so there is some overfitting. Still, the validation accuracy kept improving or stayed fairly stable, so the results were usable.

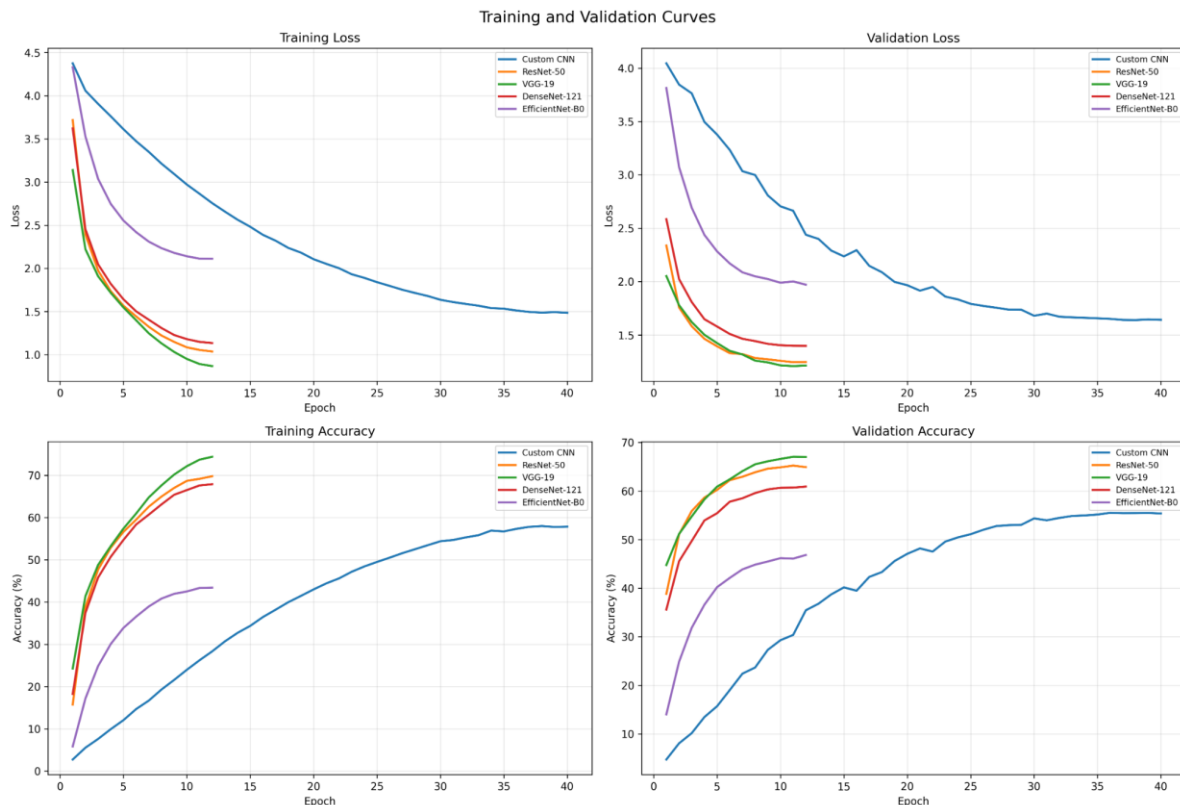


Figure 2. Training and validation loss/accuracy curves for all models.

Weight evolution in the custom CNN

To see whether the custom CNN actually changed during training, I saved filters from the first convolutional layer before and after training. At first, the filters looked like random small color patterns. After training, the changes are not dramatic, but the filters are clearly not identical anymore. This makes sense because the first layer usually learns low-level patterns such as color contrasts, edges, and simple textures.

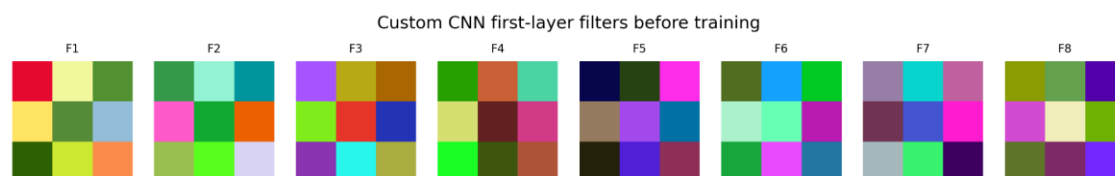


Figure 3. First-layer filters before training.

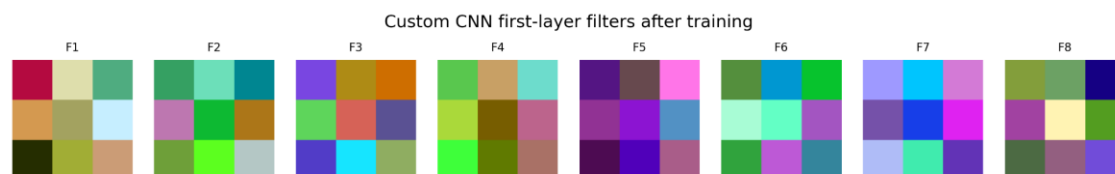


Figure 4. First-layer filters after training.

Evaluation results

The final evaluation was done on the test set. Because CIFAR-100 is a 100-class problem, TP, TN, FP, and FN were calculated with a one-vs-rest aggregation over all classes. This is why the TN values look very large: 10,000 test samples are evaluated across 100 class comparisons, giving 1,000,000 total one-vs-rest decisions.

Model	Accuracy	TP	TN	FP	FN	Macro F1
Custom CNN	55.99%	5599	985599	4401	4401	0.5531
ResNet-50	64.15%	6415	986415	3585	3585	0.6379
VGG-19 BN	66.66%	6666	986666	3334	3334	0.6642
DenseNet-121	60.57%	6057	986057	3943	3943	0.6018
EfficientNet-B0	45.88%	4588	984588	5412	5412	0.4467

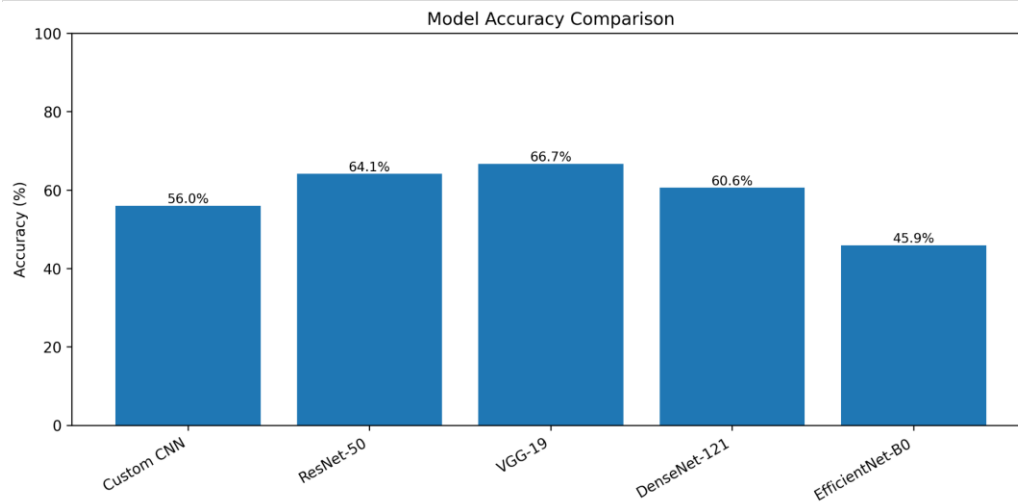


Figure 5. Accuracy comparison on the test set.

Model comparison

The best model in my run was VGG-19 BN, with 66.66% accuracy and a macro F1-score of 0.6642. ResNet-50 came next at 64.15%, followed by DenseNet-121 at 60.57%. The custom CNN reached 55.99%, which I think is a decent result because it was trained from scratch. EfficientNet-B0 had the lowest result, with 45.88% accuracy.

I was a bit surprised that EfficientNet-B0 performed the worst, because it is usually a strong model. Looking back, I think the image size probably explains a lot of this. I kept CIFAR-100 at 32x32 instead of resizing it to 224x224. EfficientNet was designed around ImageNet-sized images and uses early downsampling, so on very small images it can lose spatial information quite quickly. VGG-19 BN is simpler and uses repeated 3x3 convolutions, so it seemed to handle the small input size better in this experiment.

If I had more time, I would try the pretrained models again with 224x224 resized images and compare the result. That would take longer and use more GPU memory, but it might give EfficientNet-B0 a better chance. For this run, the results are still consistent with the preprocessing choice and the training curves.

Conclusion

Overall, both parts of the work behaved as expected. Gradient descent converged very close to the theoretical minimum. In the image classification part, the pretrained models generally performed better than the custom CNN, but the exact ranking depended on the image size and architecture. The custom CNN still learned steadily and produced a reasonable result. The main thing I noticed is that model architecture cannot be judged alone; preprocessing choices, especially image size, can change the outcome a lot.